TransformerDecoder#

https://pytorch.org/docs/stable/generated/torch.nn.TransformerDecoder.html

Description:

TransformerDecoder is a stack of N decoder layers. This TransformerDecoder layer implements the original architecture described in the Attention Is All You Need paper. The intent of this layer is as a reference implementation for foundational understanding and thus it contains only limited features relative to newer Transformer architectures. Given the fast pace of innovation in transformer-like architectures, we recommend exploring this tutorial to build efficient layers from building blocks in core or using higher level libraries from the PyTorch Ecosystem. All layers in the TransformerDecoder are initialized with the same parameters. It is recommended to manually initialize the layers after creating the TransformerDecoder instance. decoder_layer (TransformerDecoderLayer) – an instance of the TransformerDecoderLayer() class (required).

Function Signature

```
class torch.nn.TransformerDecoder(decoder_layer, num_layers,
norm=None)[source]#

Parameters

forward(tgt, memory, tgt_mask=None, memory_mask=None,
tgt_key_padding_mask=None, memory_key_padding_mask=None,
tgt_is_causal=None, memory_is_causal=False)[source]#

Parameters

Return type
```

Returns:

Tensor

Code Examples

```
>>> decoder_layer = nn.TransformerDecoderLayer(d_model=512,
nhead=8)
>>> transformer_decoder = nn.TransformerDecoder(decoder_layer,
num_layers=6)
>>> memory = torch.rand(10, 32, 512)
>>> tgt = torch.rand(20, 32, 512)
>>> out = transformer_decoder(tgt, memory)
```

Notes & Warnings

> ⚠ WARNING Warning All layers in the TransformerDecoder are initialized with the same parameters. It is recommended to manually initialize the layers after creating the TransformerDecoder instance.

Detailed Documentation

Documentation

TransformerDecoder is a stack of N decoder layers.

This TransformerDecoder layer implements the original architecture described in the Attention Is All You Need paper. The intent of this layer is as a reference implementation for foundational understanding and thus it contains only limited features relative to newer Transformer architectures. Given the fast pace of innovation in transformer-like architectures, we recommend exploring this tutorial to build efficient layers from building blocks in core or using higher level libraries from the PyTorch Ecosystem.

All layers in the TransformerDecoder are initialized with the same parameters. It is recommended to manually initialize the layers after creating the TransformerDecoder instance.

decoder_layer (TransformerDecoderLayer) – an instance of the TransformerDecoderLayer() class (required).

num_layers (int) – the number of sub-decoder-layers in the decoder (required).

norm (Optional[Module]) – the layer normalization component (optional).

Pass the inputs (and mask) through the decoder layer in turn.

tgt (Tensor) – the sequence to the decoder (required).

memory (Tensor) – the sequence from the last layer of the encoder (required).

tgt_mask (Optional[Tensor]) – the mask for the tgt sequence (optional).

memory_mask (Optional[Tensor]) – the mask for the memory sequence (optional).

tgt_key_padding_mask (Optional[Tensor]) – the mask for the tgt keys per batch (optional).

memory_key_padding_mask (Optional[Tensor]) – the mask for the memory keys per batch (optional).

tgt_is_causal (Optional[bool]) – If specified, applies a causal mask as tgt mask. Default: None; try to detect a causal mask. Warning: tgt_is_causal provides a hint that tgt_mask is the causal mask. Providing incorrect hints can result in incorrect execution, including forward and backward compatibility.

memory_is_causal (bool) – If specified, applies a causal mask as memory mask. Default: False. Warning: memory_is_causal provides a hint that memory_mask is the causal mask. Providing incorrect hints can result in incorrect execution, including forward and backward compatibility.

see the docs in Transformer.